第四部分 · 记忆的表示与存储架构

文本记忆与结构化记忆

怎样保存一条可管理的长期记忆?

Text Memory · Key–Value · JSON · Relational Memory

> 不仅要把它保存下来，还要知道：
> 它属于谁、从何而来、何时有效、如何更新。

原始文本 → 结构化字段 → 关联与版本 → 可靠利用

零基础理论讲义 · 18张原理图 · 12道思考题及详解

v1.0 | 2026-10-18 | 纯理论、无实验

《智能体记忆系统：从零基础到科研理解》 · 第17/32讲 · 理论课程

本讲核心问题： 智能体已经知道哪些事件值得记住，也知道需要保留时间、来源与版本，但这些内容最终应当以什么形式存在？保存一大段自然语言是否足够？把它拆成字段以后，系统真的就更可靠了吗？

教学约定： 全部人物、项目、日志、价格、日期与计算结果都是**教学虚构示例**，不是任何真实研究任务的记录、预算或授权。JSON、键值及表格只是帮助理解数据结构的“样本”，**本讲没有编程、实验或安装任务**。文末所列论文和官方技术文档用于解释设计思想；示意图不代表其原始架构的逐字复刻。

学习目标：从“能存下来”到“可查询、可更新、可追溯”

完成本讲以后，你应当能够：（1）独立区分文本、结构化字段与物理数据库；（2）解释原始记录、当前状态、历史版本和检索索引为什么不能随意混在一起；（3）用一个具体案例说明纯文本、键值、JSON文档和关系表的不同信息表达方式；（4）理解主键、字段、实体、关系、命名空间和有效性约束的基本原理；（5）从数据结构的角度定位“旧结论压过新结论”“同名项目串线”“来源丢失”等错误；（6）解释为什么结构化本身不等于事实正确或执行授权。

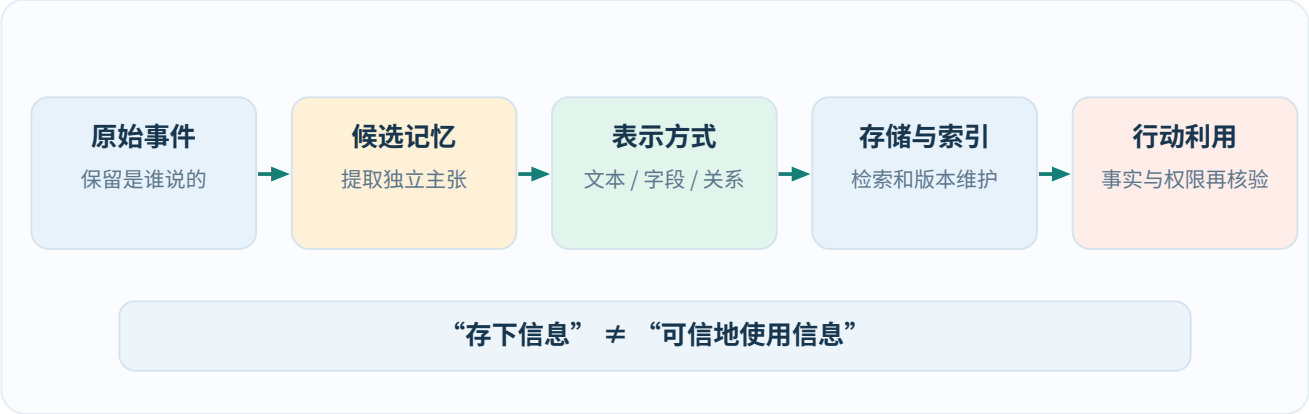


图1　从事件到行动：表示是长期记忆链路的中间层

> **本讲最重要的一句话：** 记忆表示形式决定系统更容易回答哪些问题、保留哪些关系、执行哪些更新；但是**表示结构再精细，也不自动证明内容为真、状态最新、来源可信或操作已获授权**。

一、从一个连续科研项目出发：为什么一句话越来越不够用？

设想一个虚构的科研助手负责协助项目 P（一个智能体可靠性课题）：

- **第3天：** AI提出“也许可以调用100次收费API检验一个假设”。这是**建议**，不是决定。
- **第8天：** 研究者明确说“本阶段不要新增收费API调用，先进行离线分析”。这是指定项目和阶段的**当前约束**。
- **第11天：** 某工具返回`HTTP 504`。这是一条**观测结果**，尚不能证明问题出在模型本身。
- **第19天：** 查看网关日志后确认超时与基础设施有关。这是一项有证据的**归因修订**。
- **第30天：** 用户询问：“我们能否继续收费测试？第11天的失败说明什么？”

如果长期记忆仅保存一份自然语言摘要：“项目P曾讨论100次收费API调用，发生过超时，后来做了复盘”，它可能有帮助，却不足以独立回答：谁提议？谁决定？哪条决定目前生效？超时最初是什么现象？后来的因果归因有什么证据？

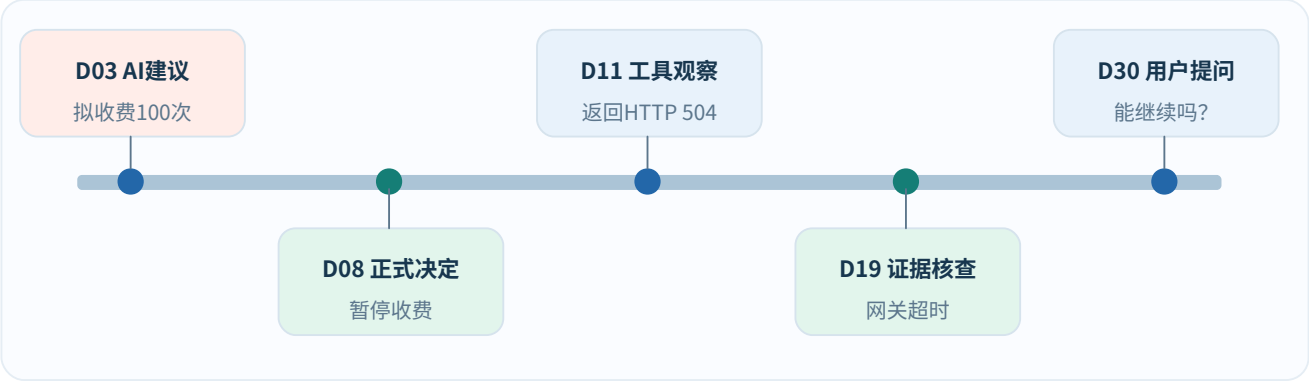


图2　同一项目的五个时间节点，包含五种不同的主张类型

这一连串问题不是“多存几千个Token”就能自动解决的，而是要求**同一记忆中多个信息维度可被明确地区分、定位和关联**。由此产生不同表示方式的设计选择。

二、首先分清三个层面：内容、表示与存储

2.1 第一层：内容——智能体究竟记住了什么

内容可以是完整聊天、一次工具调用、一个项目决定、某项失败经验，也可以是知识抽象。它回答的是**记忆的语义**。例如“项目P当前暂停新增收费调用”属于内容，且其中“当前”和“项目P”是不可丢失的限定条件。

2.2 第二层：表示——计算机如何组织这些内容

表示可以是一句话、一个键和值、一组JSON字段、一张关系表中的几行记录，或者实体与关系组成的图结构。它回答：**怎样把同一件事拆成机器更容易访问和检查的单元？**

2.3 第三层：物理存储——这些数据实际放在哪里

文本可以存在文件、对象存储、SQLite或PostgreSQL；JSON也可以存在文件、关系数据库的JSON字段或文档数据库；同一份结构化记录还可以生成全文、向量等派生索引。**“结构化”描述组织形式；“数据库类型”描述存储与查询机制。二者不是同一分类维度。**



图3　先分层：内容、表示形式、物理保存与访问路径

这也意味着：把聊天记录放进数据库，不意味着它已经变成经过筛选的长期记忆；把一句记忆转换为JSON，也不意味着它自动拥有更好的长期推理能力。

三、第一种表示：自然语言文本记忆（Text Memory）

3.1 它的基本思想是什么?

最直接的方式就是写一段人能读懂的说明，例如：

> **项目P · 第8天决策摘录：** 研究者要求本阶段暂停新增收费API调用，优先进行离线分析。第3天AI提出的“100次付费调用”仅为未获批准的建议。此条内容来自研究者第8天的原始回复，可在项目P的历史回复中核查。

这一段文本同时提供了事实、背景、时间和来源。**文本并不是“无结构”的代名词**：它可以有标题、分段、项目标签、日期、引用和固定模板。只是许多关系仍由自然语言隐含表达，程序不能毫无歧义地直接读出。

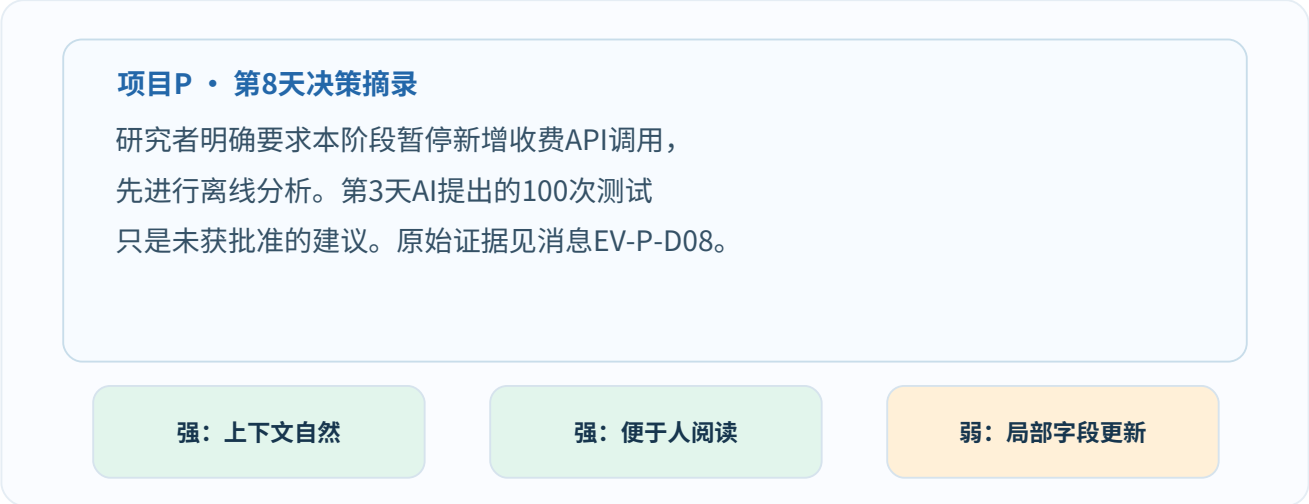


图4　自然语言备忘录：保留来龙去脉，但精确字段藏在句子里

3.2 文本有什么优势?

文本适合保留复杂背景、事件因果、解释、条件和尚未形成清晰字段的经验。比如“第11天发生超时，但当时不能判定责任，直到第19天才通过网关日志进行归因”，如果压成`error=504`，就失去了整个认识演化过程。

它也容易被语言模型阅读和总结，适合低成本原型、项目笔记、个人经验卡和反思记录。真正的优势在于**表达丰富、容易检查原话、可以容纳例外情况**。

3.3 文本有什么局限?

设想要寻找所有“当前暂停付费调用的项目”。如果上千篇自然语言笔记表达各不相同，有的写“禁止调用”，有的写“暂不执行”，还有的写“先不用付费API”，单靠字符串匹配会漏掉很多；语义检索虽可增加召回，也可能把“建议恢复”误认成“已决定恢复”。

另一个难点是**局部更新**：如果一句话同时讲第3天建议、第8天决定、第11天超时、第19天归因，之后只修改“预算状态”可能不小心重写其他事实。只保存最新摘要，还可能失去历史原话。

3.4 文本并不排斥元数据

一份文本记忆仍然可以带`memory_id`、项目ID、创建时间、来源消息ID及状态标签。所谓“文本记忆”通常是指**主要语义内容以自然语言表达**，不表示系统没有字段，更不表示只能用纯文本文件存储。此点贯穿本讲所有比较。

四、第二种表示：键值记忆（Key–Value Memory）

4.1 从一个“带标签的抽屉”理解键值

想象办公室有许多抽屉，每个抽屉门口有一个唯一标签（Key），打开后看到其内容（Value）。例如：

- `project:P:goal` → “研究智能体可靠性”
- `project:P:paid_api_policy` → “本阶段暂停新增收费调用”
- `project:P:last_confirmed_failure` → “第11天504；第19天归因为网关超时”

注意这里的冒号只是一种便于阅读的**教学命名约定**，并非行业统一标准。键值模式强调：**已知一个确切的键时，能够快速找到对应值。**



图5　键值抽屉：用明确的键找到当前值，但要注意历史缺口

4.2 键值适合什么?

对于稳定、明确、经常按标识符读取的事实很合适，比如某项目的当前阶段、用户已确认的特定偏好、某执行器版本或任务状态。使用固定键，应用可以明确表达“我要当前项目P的收费策略”，不必先在大段自然语言中寻找。

但**键名需要命名空间**。如果只有`paid_api_policy`，项目P和项目Q可能互相覆盖；若同一系统服务多个用户，必须保留租户、用户、项目等隔离边界。键值数据库是否快速和稳定，还取决于实际实现、索引、并发与持久化配置。

4.3 一键一值会丢失什么?

只保存`project:P:paid_api_policy = blocked`，很难知道是谁设定、从何时生效、是否有例外、此前是什么状态、哪个消息是原始依据。**解决办法不是否定键值，而是让Value成为富结构对象，或让它链接到版本与来源记录。**

另一个局限是复杂查询：要找“所有在某段时间内由特定负责人确认、且与工具故障有关的约束”，单个键无法天然表达所有筛选关系，往往需要额外索引、扫描或换一种表示。

4.4 读写的动态过程

一个安全的教学流程：根据用户和项目生成隔离键 → 查询是否已有当前值 → 验证新事件能否改变该值 → 更新Value/创建新版本 → 保留旧值与证据 → 刷新相关索引/缓存。注意，**同一键被成功覆盖不意味着这次覆盖是语义正确的**；权限与事实判断必须发生在写入前或写入控制路径中。

五、第三种表示：结构化对象与JSON文档

5.1 “对象”让一条记忆同时拥有多个可命名属性

与一个键只对应一句话相比，结构化对象把内容拆成明确字段。例如以下是**声明式数据样本，不是需要运行的程序**:

```
{
  "memory_id": "M-P-008",
  "project_id": "P",
  "kind": "project_decision",
  "claim": "本阶段暂停新增收费API调用",
  "polarity": "prohibit",
  "status": "active",
  "speaker_role": "project_owner",
  "valid_from": "D08",
  "source_event_id": "EV-P-D08-03",
  "supersedes": null,
  "scope": {"project": "P", "stage": "current"}
}
```

左边的名称（如`status`）是**字段名**，右边是对应取值。`scope`内部还有一个子对象，展示“对象可以嵌套对象”。这里的D08是教学时间标记，不是实际日期字符串规范。



图6　结构化记忆卡：一条主张拆成可独立核对的字段

5.2 它比纯文本强在哪里?

系统可以直接查找`status=active`、`project_id=P`、`kind=project_decision`等字段，而不必每次请模型从整段文字中猜测。也更容易设置字段是否必填、枚举值、数据类型和基本校验，例如`valid_from`应该是时间而非一段无意义叙述。

显式结构让检查更容易，不代表它会自动被正确填充。 模型可能把第3天AI建议错标成`project_decision`，JSON仍完全合法。**“格式正确”与“语义正确”必须分别检查。**

5.3 JSON的灵活性与代价

JSON（JavaScript Object Notation）是一种常见的数据交换格式。对象字段可以嵌套、数组可以包含多条关联证据，适合表示不同类型的记忆卡。但若一条记忆叫`valid_from`、另一条写`startTime`，或者`status`的取值在不同模块中不统一，后续查询就容易混乱。

因此较成熟的系统往往会定义**模式（Schema）**：说明哪些字段存在、数据类型是什么、哪些可空、哪些枚举值合法，以及字段之间需要满足哪些业务约束。JSON Schema等机制可描述一部分结构要求，但无法凭空验证“AI建议是否真的是用户批准”。

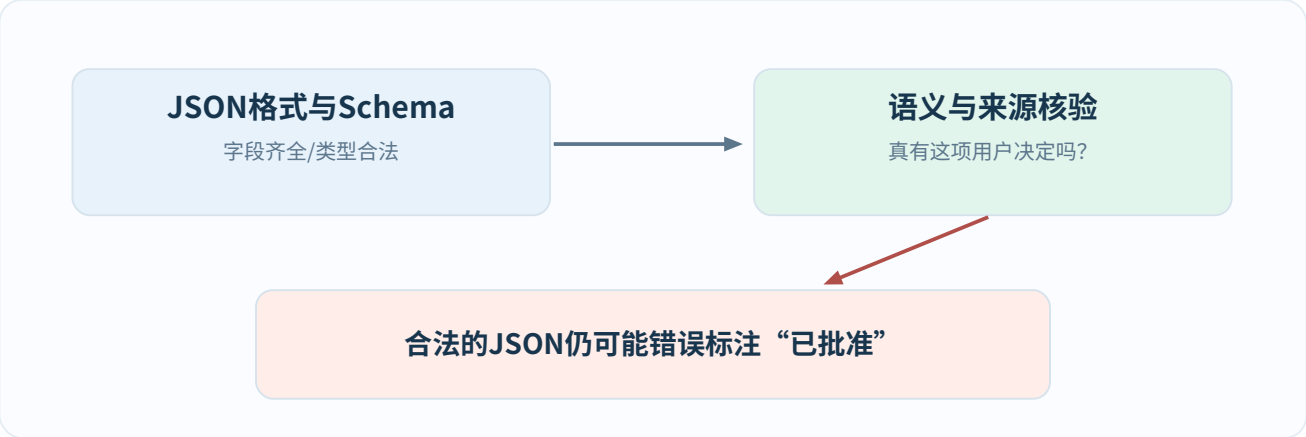


图7　格式校验和事实核验是不同关口

5.4 “缺失”“未知”“否定”不能混为一谈

看一个关键例子：`approved=false`表示已知未批准；`approved=null`可能表示尚未核实；字段完全不存在可能表示该对象没有设计或没有记录此信息。真实工程中这些含义需要明确规定，不能假设`null`永远等于“未知”。

对科研助手尤其重要：**“没有找到付费授权记录”不能被推理成“已获批准”，更不能因为缺少否定字段就默认允许执行。**执行权限需到真实授权系统查询，记忆中的状态字段只构成提示或证据线索。

5.5 一条记忆不是一个不可改变的整体

如果第19天把故障从“原因未知”更新为“网关超时”，并不应该悄悄覆盖第11天的原始现象。可以将`observed_failure`记录保持不变，新建`diagnosis`记录，并使用`based_on=EV-P-D11`或`supersedes`等关系说明当前理解如何变化。这种**分离事件与认识**的设计，使系统能够回答“第12天当时我们知道什么”，而不是仅返回后来修订过的答案。

六、第四种表示：关系表（Relational Tables）

6.1 用几张表替代一个不断膨胀的大对象

当项目增多、事件增多、需要多人协作或希望精确追踪来源时，可以把不同性质的数据放入不同表中。一个教学设计例子：

事件表 Events 保存`event_id`、`project_id`、`actor`、`occurred_at`、`raw_text`；**记忆表 Memories** 保存`memory_id`、`project_id`、`kind`、`claim`、`status`；**关系表 Links** 保存`from_id`、`relation`、`to_id`。每行都有可标识的ID。



图8　关系表示：Events、Memories与证据关联表职责分离

6.2 主键与外键为什么重要?

主键（Primary Key）好比每个人独一无二的编号，让系统知道“究竟是哪一条记录”。外键（Foreign Key）让一张表的字段指向另一张表中存在的记录，例如`Memories.source_event_id → Events.event_id`，形成可追溯的关联。

但注意：数据库可以验证“被引用的事件ID是否存在”，却不能保证“这条事件确实支持该记忆结论”。后者仍需要语义核验。外键保证**引用存在与结构一致**，不自动保证**引用正确与科学有效**。

6.3 多对多关系和历史版本

一个结论可能由3条独立日志支持；同一条日志也可能支持2项不同的断言。如果只在记忆里保存一个`source_event_id`，就不足以表示多对多关系。可以增加`MemoryEvidence(memory_id,event_id,support_type)`这样的关联表，保存每一条支持或反驳关系。

历史更正也是如此。`M10 SUPERSEDES M07`表示M10替代M07的某个适用范围内的结论；这不要求删除M07，也不意味着每一项旧信息都为假。

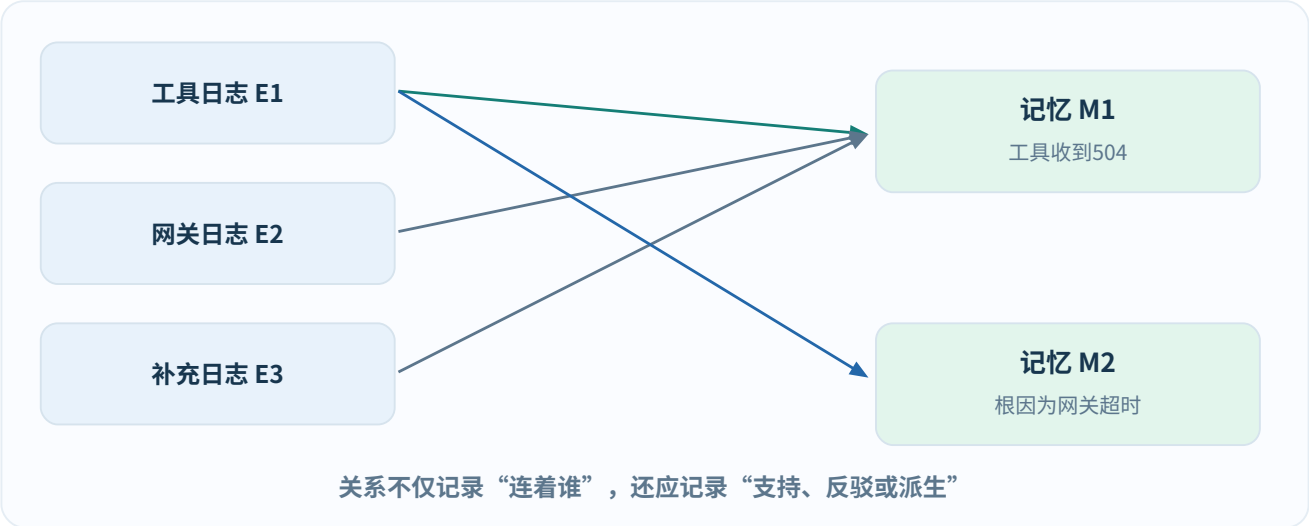


图9　一条证据支持两项主张，一项主张依赖多条证据

6.4 关系表、SQL与数据库不是一回事

关系模型是一种表示和约束方式，SQL是常见的查询语言，PostgreSQL等是可实现它们的数据库系统。由于不同数据库对JSON、全文检索、事务隔离和索引的支持不同，不能把特定实现视为一切关系数据库的固有性质。

对于零基础理解，记住“**表负责把对象分门别类，ID负责精确定位，外键与关联表负责建立可检验的引用**”就足够了。暂时不需要记SQL语法。

七、实体关系与图：不是所有联系都适合塞进一个字段

除存储单条事实，长期研究记忆还需要表示“谁提出、谁否决、哪个事件支持哪个结论、哪个决定替代了哪项旧决定”。这些都是**关系**。可以用表中的关联记录表示，也可以用图数据结构表示；图模型强调**节点与带类型的边**。

例如：`AI建议D3 —proposed_by→ Agent-A`，`决定D8 —issued_by→ 研究者`，`决定D8 —supersedes→ 项目P旧策略`，`归因D19 —supported_by→ 网关日志`。这些边比“项目P有个记忆列表”表达了更丰富的语义。

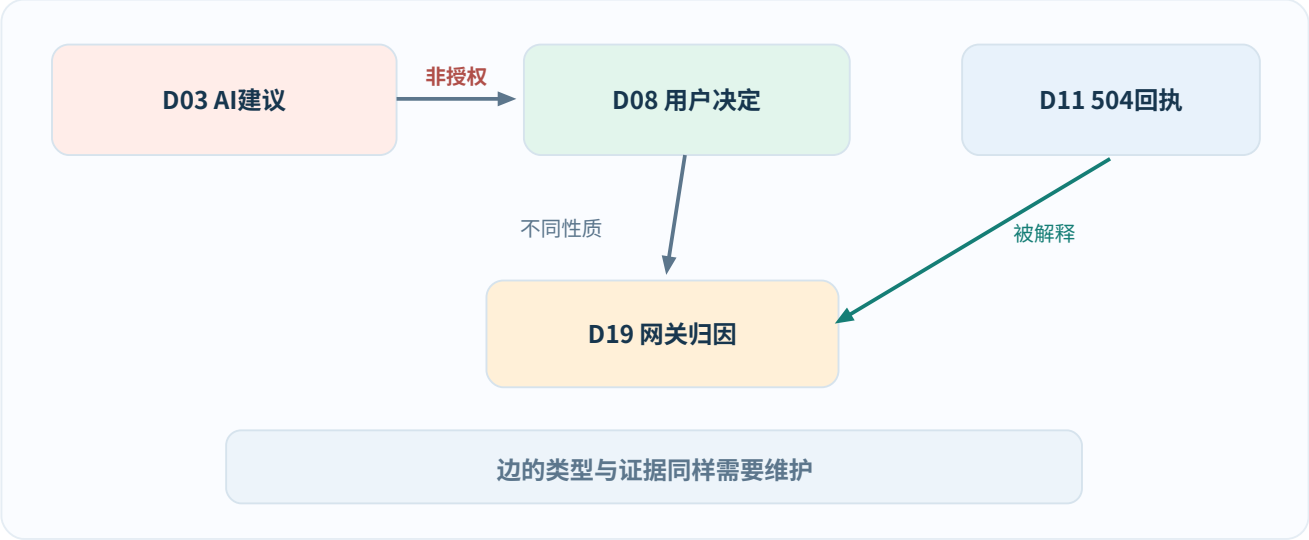


图10　一幅关系图：谁建议、谁决定、什么证据支持哪项归因

重要边界：　画出关系图并不等于拥有“图数据库”；图关系也可以存于关系表。更不意味着图上的边都是真实的。边仍需来源、时间、置信与可能的撤销机制。A-MEM使用动态的记忆链接与结构化属性组织笔记；Mem0进一步研究记忆提取、合并、检索及图式表示，是理解架构方向的代表性研究，而非“唯一正确”的记忆格式。[文献1、2]

八、一个系统往往同时保留多种表示

同一条记忆可以有多个互不冲突的“视图”：

- **原始事件**：第8天用户说的完整话与事件ID。
- **当前有效状态**：项目P的收费调用政策为暂停。
- **可读文本摘要**：说明政策原因、背景、限制和出处。
- **结构化记录**：`project=P; status=active; kind=policy`等字段。

- **派生索引**：全文词项索引、语义向量或关系链接，用于检索候选。

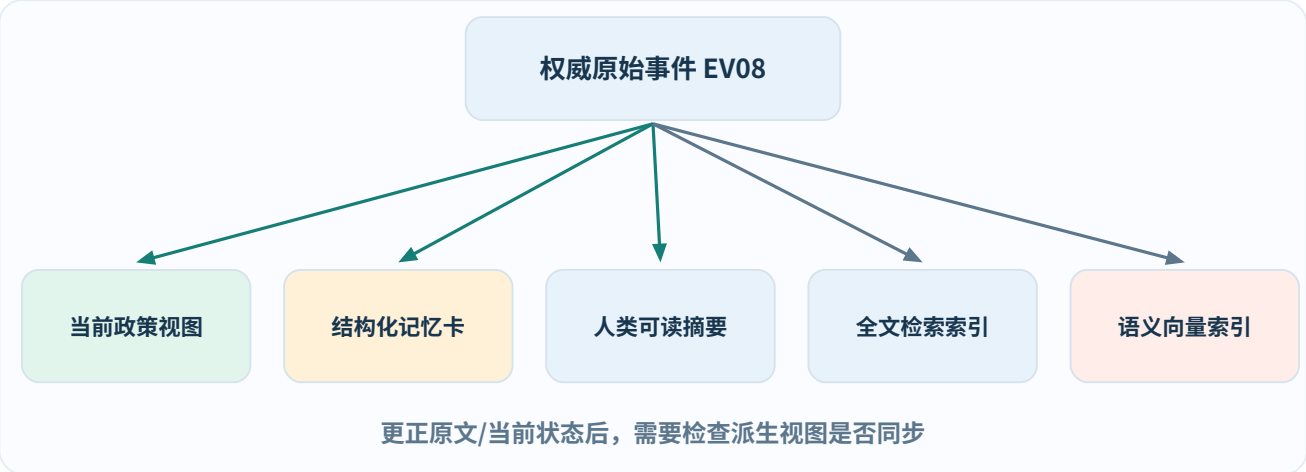


图11　同一事实的不同视图，不是五个独立事实

关键是不要把五份内容都当成五条相互独立的事实。索引只是原始记忆的**派生访问路径**；摘要通常是派生解释；当前状态是一种**基于有效规则和事件建立的视图**。必须说明哪些记录是权威来源，哪些仅便于检索，以及更正时如何保持同步。

九、从问题出发理解查询：三种问题，需要三种定位方法

9.1 “项目P当前是否允许收费？”

这是一个带明确项目ID和状态要求的查询。理想路径为：定位项目P→查当前生效约束→检验版本、时间、来源→对照真正的授权机制。语义最相似的记录不一定是当前有效的记录。第3天“建议收费”的向量可能很相关，但其状态是未批准。

9.2 “上次工具504是怎么回事？”

这是事件与诊断的联合查询。需要先找到第11天工具回执，再找到第19天网关日志的归因记录，并核实它们之间的证据关系。单靠`error=504`的键值，不能完整回答原因演化过程。

9.3 “我们以前对收费测试讨论过什么？”

这是历史时间范围的开放式查询。更适合检索多个事件与摘要，并呈现“建议→决定→后续更新”的时间线。不能只返回最新值，否则可能丢失用户要追问的历史经过。

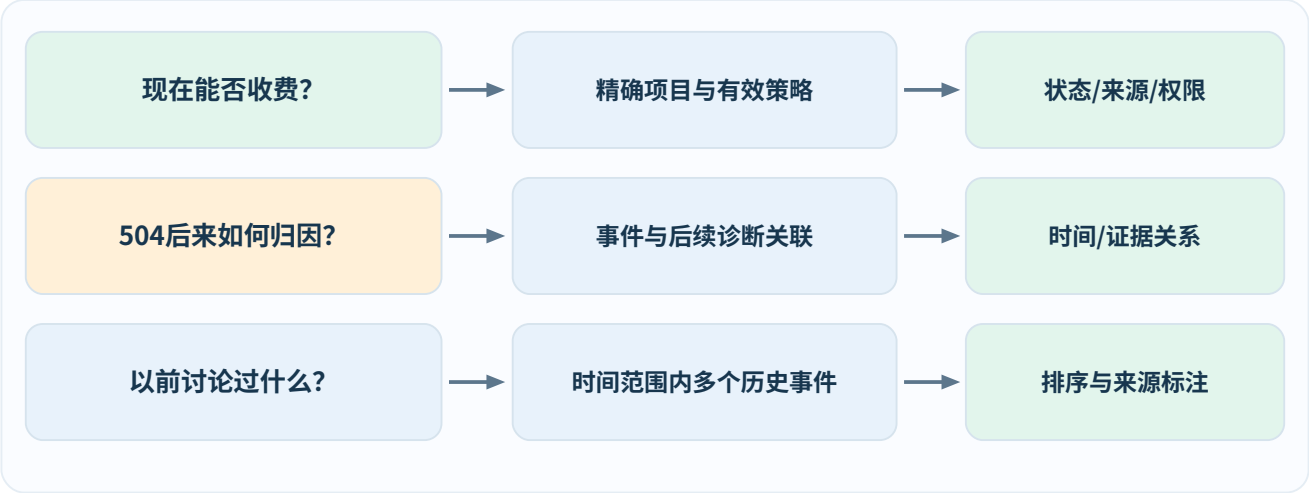


图12　同一长期助手会遇到三种不同查询路径

同一存储系统可以支持这三种问题，只是访问路径与校验策略不同。**架构设计不是先选一个时髦数据库，再强迫所有问题都用同一种检索方式回答，而应先分析任务要求。**

十、索引解决“找得到”，不自动解决“信得过”

当数据变多时，遍历所有记录会慢，因此建立索引：

1. **ID索引**：已知`memory_id`时精确定位。
2. **字段索引**：筛选`project_id=P`、`status=active`等属性。
3. **全文索引**：利用词项查找含“超时”“网关”的文本（前面已经学过倒排索引、BM25）。
4. **向量索引**：寻找语义相关的候选，例如“收费调用是否暂停？”也能匹配“本阶段不再发生新API费用”。
5. **关系遍历**：从某个结论找到其来源、反例或后续版本。

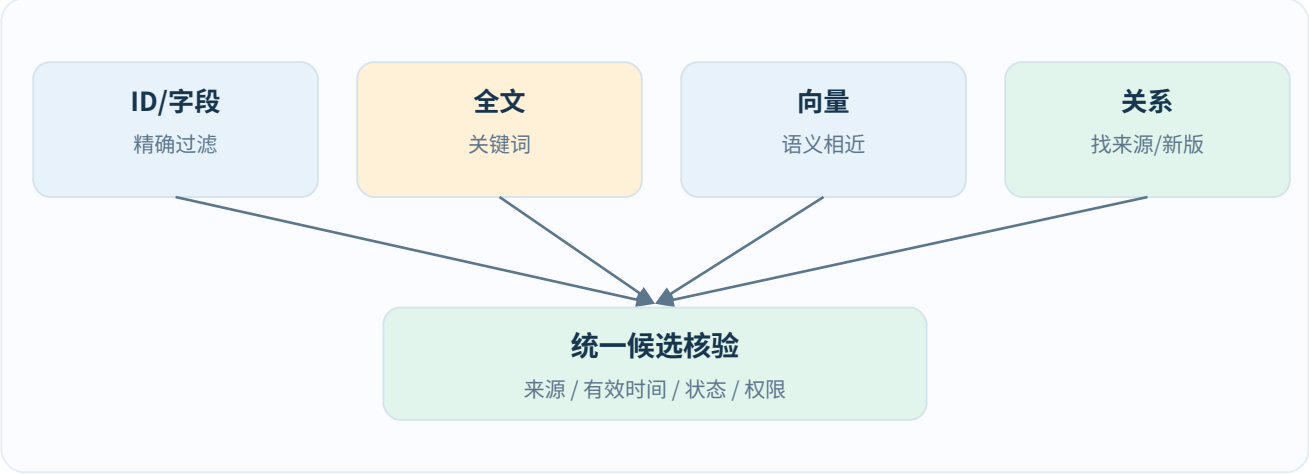


图13　检索索引的分工：多路候选最终仍需事实与权限核验

PostgreSQL官方文档说明JSONB可配合GIN等索引查询键及键值；SQLite官方FTS5资料说明如何提供全文索引。它们表明**一种数据库能支持不止一种查询方式**。[文献3、4] 但需要注意，不同索引可能与主记录异步更新；“数据库里已经写入新决定”不保证每个缓存和向量索引马上反映新决定，这属于索引一致性与系统维护问题。

十一、记忆更新为什么必须考虑时间、版本与冲突?

11.1 覆盖旧值是一种操作，不是历史知识管理的完整方案

假设第8天政策是“暂停收费”，第25天用户后来明确批准一个**限定为10次、总额不超过20元**的试验。若只把`paid_api_policy`改成`allowed`，会丢失适用范围与限额；若不更新旧值，又会一直错误拒绝经批准的实验。

更合理的方式是记录两条有效范围不同的决定：旧决定仍保留历史；新决定具有来源、起止时间、范围、上限和显式覆盖关系。**受限许可不是一般化的无限许可。**

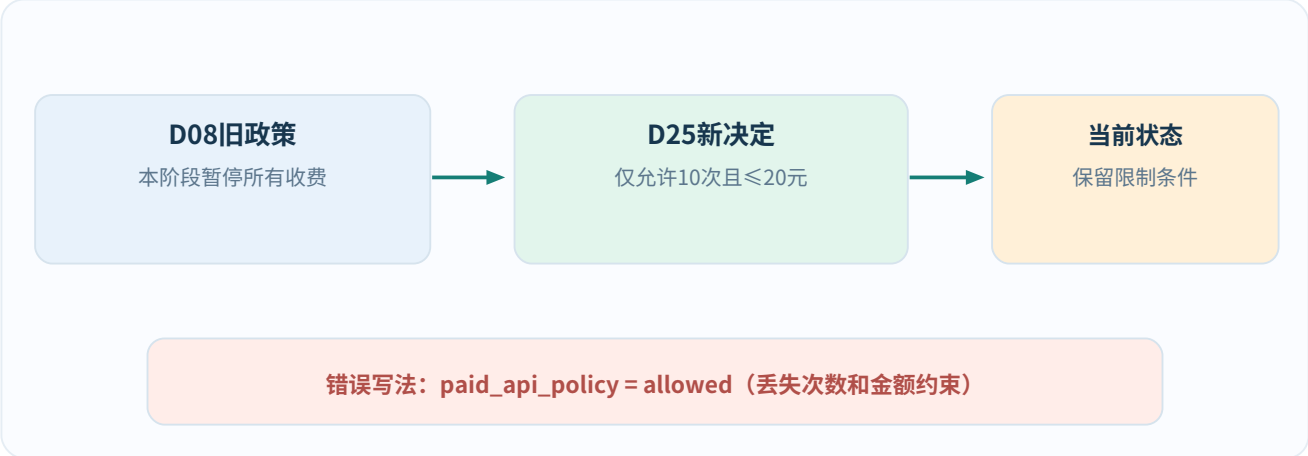


图14　政策更新：有限授权不能覆盖成“无限允许”

11.2 两个Agent同时修改怎么办?

设两个Agent都读取版本v2：Agent-A准备把事实改成v3a，Agent-B准备把同一事实改成v3b。若系统无版本检查，最后一次写入可能覆盖先前更正，造成“丢失更新”。

乐观并发控制（Optimistic Concurrency Control）的直观思想是：**写入前检查旧版本是否仍然是自己看到的版本**，若已经变化就重新读取、合并或报冲突。它能防止一类并发覆盖问题，但不能自动判断两个自然语言结论谁正确。

11.3 删除记忆也需要明确语义

“删除当前可见摘要”“标记某条结论失效”“从检索索引排除”“按隐私要求清理保存的数据”不是同一操作。一个系统可以保留历史审计，但也必须遵守适用的数据保护、删除与访问规则；**不能借口审计而无限保留不该留存的敏感信息**。

十二、隔离边界：项目P的记录为什么不应跑到项目Q?

假设一个智能体同时服务多个项目甚至多个用户。即使两条记忆在语义向量上非常相近，也不代表项目Q能够访问项目P的私有数据。检索流程的第一步应先确认**身份、租户、项目、访问范围**，再选择有权查看的候选记忆。



图15　同名项目与同名键不能跨用户泄露记忆

命名空间（Namespace）可以理解为抽屉所属的“柜子”：同名键只有在某个用户或项目范围内才有意义。但是字符串命名空间本身并不等于安全访问控制；真正的隔离还要由认证、授权、查询过滤、数据访问策略等机制落实。

这与第16讲的来源追溯不同：来源链回答“从哪里来”，权限边界回答“谁能看、谁能改、谁能执行”。两条链可能相关，却不能互相替代。

十三、如何比较文本、键值、结构化文档与关系表示？

表示方式	最擅长的事情	常见难点	适合的记忆角色
自然语言文本	保存丰富上下文、解释、事件细节	精确字段筛选和小范围更新困难	原始事件、反思、可读摘要
简单键值	按稳定键读取当前值	历史与复杂关系常需扩展	当前阶段、简单偏好、明确配置
JSON等结构化对象	一条记忆内包含多字段、数组和嵌套结构	Schema漂移、填充错误、嵌套更新	元数据丰富的记忆卡
关系表	规范化、ID引用、多对多关系、条件查询	拆表与关联维护存在开销	大量实体、来源、版本与审计关系
图式表示	强调实体和关系、多跳链接	边的来源与动态维护困难	知识关联、依赖与版本图

以上是**典型设计取舍**，不是不变的性能排名。关系数据库能保存JSON、全文及向量索引；键值存储的Value也可以是结构化对象；文本的格式也可以高度统一。**不同方案并非相互排斥，常见做法是混合使用。**



图16　五类表示方案的典型侧重点（并非绝对性能排行）

13.1 贯穿案例：你应如何为科研助手选择表示？

若任务只有“记住我喜欢中文回答”，一条文本或带范围的键值就可能足够。若任务要区分多个课题的预算、证据、冻结条件、原始运行日志和历史决策，就更需要结构化字段与关联关系；而复盘中复杂的因果与失败条件仍应保留自然语言原文。

复杂度必须由任务需要证明，而不是因为图数据库、知识图谱或向量库看起来先进。 能用简单结构忠实回答问题时，无须先构建高成本系统；简单方案不能满足来源、更新或多跳需求时，再增加必要结构。

13.2 两个容易被忽略的成本

维护成本： 结构越精细，越需要维护类型、状态同步、迁移规则及异常处理。**推理成本：** 文本很灵活，却可能在每次查询时都要求LLM重新从长段中解析事实；结构化记录减少部分解析，但并不能消除推理阶段的语义核验。

因此要从**长期生命周期成本**理解设计，而不是只比较“这一条记忆占多少字节”。

十四、让整个模型在脑中动起来：第30天到底怎样回答？

现在重放项目P的完整查询：

用户提问： “第11天504的原因是什么？我们能继续付费测试吗？”

第一步：明确范围。 当前用户、项目P、需要查询历史事件和当前有效约束。**第二步：找到原始事件。** 第11天记录工具返回504，事件本身不含明确根因。**第三步：寻找后续诊断。** 第19天的网关日志支持“基础设施导致超时”的结论，同时保留它依赖哪份证据。**第四步：读取有效策略。** 第8天用户要求本阶段停止新增付费调用，未找到在当前作用域内覆盖此决定的新授权。**第五步：组织回答。** 先解释工具504的观察与后来的归因，再明确需要新授权才能进行付费操作。**第六步：保留本轮反馈。** 如果用户后来作出新决定，生成新的事件和版本而不是静默改写历史。



图17　第30天的可靠问答：从当前问题一路追溯到有效来源

注意这不是某个“完美算法”的保证。每一步仍有可能因为来源缺失、误提取、索引滞后、模型推理错误或权限控制失效而出问题。真正可靠的系统需要观察完整链路。

十五、本讲与第18讲的关系：从“记忆表示”走向“记忆检索架构”

本讲讲的是**一条记忆怎么被表达、怎样保留关联和版本**。下一讲将进一步解释：当有成千上万条记忆时，如何通过向量索引、混合检索与数据库的组合，在时间和资源约束内找到真正相关的候选。二者必须配合理解：表示会改变可检索属性，检索策略也会反过来要求表示保留足够的字段。

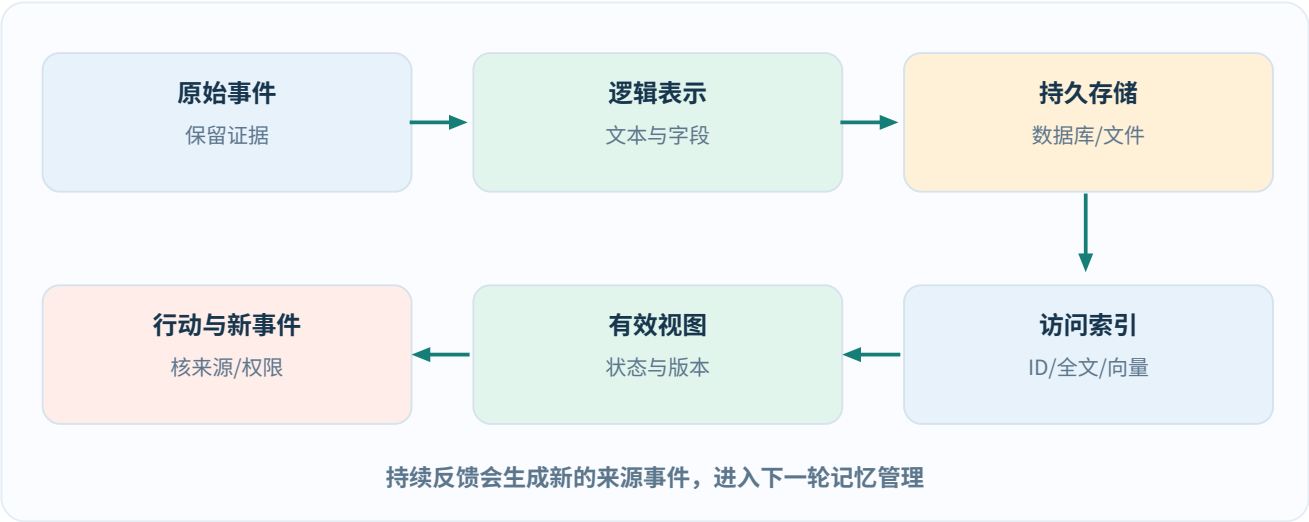


图18　本讲知识地图：表示、存储、检索、版本和决策构成闭环

十六、零基础常见误区与纠正

1. **误区：文本记忆就是没有结构。** 纠正：文本也可有标题、字段、引用和规范模板，差异在语义表达主导方式。
2. **误区：JSON放进数据库就是真正的长期记忆。** 纠正：它首先是数据对象；是否筛选、更新、检索和正确利用要另看系统机制。

3. **误区：结构化一定比文本准确。** 纠正：模型可能以合法字段存入错误值，导致“结构上正确、语义上错误”。
4. **误区：一条记忆只需要一个值。** 纠正：对于长期任务，时间、来源、范围和版本可能同样不可缺。
5. **误区：只要有来源ID就已经证实。** 纠正：来源ID只说明可以追溯；还要核对它是否支持相关主张。
6. **误区：有了关系表就不用自然语言。** 纠正：复杂反思、例外和解释仍需文本，混合表示常更合理。
7. **误区：图记忆必须购买图数据库。** 纠正：图是表示模型，关系表也可以保存节点与边。
8. **误区：向量相似最高就是当前有效事实。** 纠正：相似性只是候选信号，必须检验状态和时间。
9. **误区：`approved=null`等于`approved=true`或`false`。** 纠正：缺失、未知与明确否定需要独立约定。
10. **误区：数据库事务自动保证跨缓存、向量库、文件库的原子一致。** 纠正：单一数据库事务不自动覆盖所有外部组件。
11. **误区：最后写入一定最新、一定对。** 纠正：写入时间不等于事实有效时间，更不等于来源权威性。
12. **误区：只要命名空间不同就绝对安全。** 纠正：命名空间是组织维度，安全还需独立访问控制。

十七、理论思考题（先独立推演）

题1： “文本记忆、JSON、PostgreSQL、向量索引”属于同一分类维度吗？请逐个说明。

题2： 项目P的文本记忆写成：“AI第3天提出100次付费调用，第8天用户暂停付费。”只改成`project:P:paid_api_policy=blocked`以后，分别保留和丢失了什么信息？

题3： 为什么`memory_id`与`source_event_id`应当是不同的ID？能否举出一条事件支持多个记忆断言的情况？

题4： 教学记录`approved=null`，能否得出“可以开始付费调用”的结论？你会如何区分“未知”与“明确未批准”？

题5： 第11天返回504，第19天确认网关超时。应当直接把第11天`error=504`改写成`error=gateway`吗？为什么？

题6： 10条记忆都有关收费，其中9条旧建议的语义很接近当前问题，只有1条最新用户正式决定。为什么单纯Top-1向量检索可能误导智能体？

题7： 一条结论依赖3份工具日志，而一份日志又支持2条结论，使用一个`source_event_id`字段有什么问题？

题8： 两个Agent同时读取了版本v4并想修改同一条记忆，其中一个已经提交v5。另一个应当怎么处理？这是否能保证语义正确？

题9： “用户A的项目P”和“用户B的项目P”都存在`paid_api_policy`，数据库应如何防止串线？单纯使用`P`足够吗？

题10： 相比只保存当前结论，保存历史事件与当前状态两个视图，会获得哪两种不同的回答能力？

题11： 任务X主要是私人偏好记忆，任务Y是长期科研实验授权与Trace追溯。你会对两者选择完全相同的存储表示吗？请说明复杂度取舍。

题12： 某系统说“我们有关系表、JSON和向量库，因此记忆完全可靠”。至少列出四种仍可能发生的失效。

十八、参考答案与详细推导

第1题： 不属于。文本是主要**语义表示**，JSON是一种**结构化交换/表示格式**，PostgreSQL是具体**数据库管理系统**，向量索引是**访问路径/检索结构**。同一文本可以保存在PostgreSQL，同一JSON文档也可生成向量索引。这正是“内容—表示—存储—索引”四层划分的意义。

第2题： 新键值保留了项目P当前收费策略这一**单一状态**，查询更直接。但丢失了第3天AI建议的历史、第8天是谁宣布暂停、原始证据位置、有效期、阶段限制及撤销机制。可以继续把值丰富为带元数据的结构化对象，并关联原始事件。

第3题： `memory_id`标识经过提取与整理的**记忆断言**，`source_event_id`标识真实发生或被记录的**原始事件**，生命周期不同。第8天用户一段原话可能同时明确“暂停付费”和“先离线复盘”，产生两个独立记忆卡，却共享一个来源事件ID。混为同一ID会影响版本、引用和修订。

第4题： 不能。`null`可能表示未知、不适用或未填写，具体含义需Schema约定；不代表允许执行。明确未批准应有可核验的否定决定或现行策略；没有授权记录时，执行控制器不应据此做收费动作，须查询真正授权来源。

第5题： 不应直接覆盖原始504事件。504是**第11天观察到的响应**；网关原因是**第19天形成的后续诊断**。可以新增诊断记忆并关联第11天日志，使系统既能回答“当时观察到什么”，也能回答“后来确认原因是什么”。

第6题： 因为语义相似并未编码“谁有权决定、何时生效、是否被后续决定覆盖”。Top-1只是最像查询的问题片段。应先限制项目与用户作用域，结合状态/有效时间找候选，再回查当前授权和证据；向量检索是辅助，不是决策最终裁判。

第7题： 一个字段只能表达一个直接来源，无法保存**多对多支持关系**及支持/反驳性质。应使用证据列表或关联表，例如记忆M1连接E1/E2/E3；事件E1同时连接M1/M2。并且要记每条边的证据支持类型，避免引用存在却不支持主张。

第8题： 后提交者应该发现期望版本v4与当前版本v5不符，暂停直接覆盖，重新读取并判断如何合并或记录冲突。这是**乐观并发控制**的直觉。它只解决写入竞态，不能自行裁决哪个自然语言判断真实。

第9题： 至少需要`tenant_id/user_id + project_id`等稳定复合命名空间和真实认证后的访问过滤，必要时还要行级授权或独立隔离存储。单纯`P`不能全局唯一识别一个用户的项目；同名字符串也不能成为安全认证凭据。

第10题： 历史事件用于回答“过去发生过什么、那时知道什么”，当前状态用于回答“现在对该任务仍然有效的约束是什么”。只有历史而无有效状态容易每次重新推理；只有当前值而无历史则无法审计与复盘。

第11题： 不必相同。X可先用简单文本或带范围的键值，但要关注隐私和偏好变化；Y通常需要明确的事件ID、版本、权限状态、来源链接、多对多证据关系以及原始Trace，可能更适合**文本+结构化字段+关联表**的混合方式。只在任务需求真的要求时才支付额外架构成本。

第12题： 至少包括：①写入时把AI建议误分类成用户决定；②JSON虽然格式合法却写错否定词；③向量检索找回过时建议；④缓存/索引未同步；⑤来源证据不能支持结论；⑥多租户过滤错误泄露私有数据；⑦权限控制器错误；⑧模型在正确上下文中依然推理错误。数据库的存在不会自动消除这些错误。

十九、本讲知识地图与衔接

记忆系统先接收有来源的事件 → 选择保留自然语言原文和必要字段 → 用稳定ID将事件、主张、版本关联 → 根据任务构造当前状态视图 → 建立全文/向量/字段等访问路径 → 用检索与过滤找到候选 → 回源核验事实、时间和权限 → 执行前由控制器独立判断 → 新反馈再次形成事件与版本。

独立重建这条链路，你已经具备进入下一部分“存储架构与检索机制”的理论基础。

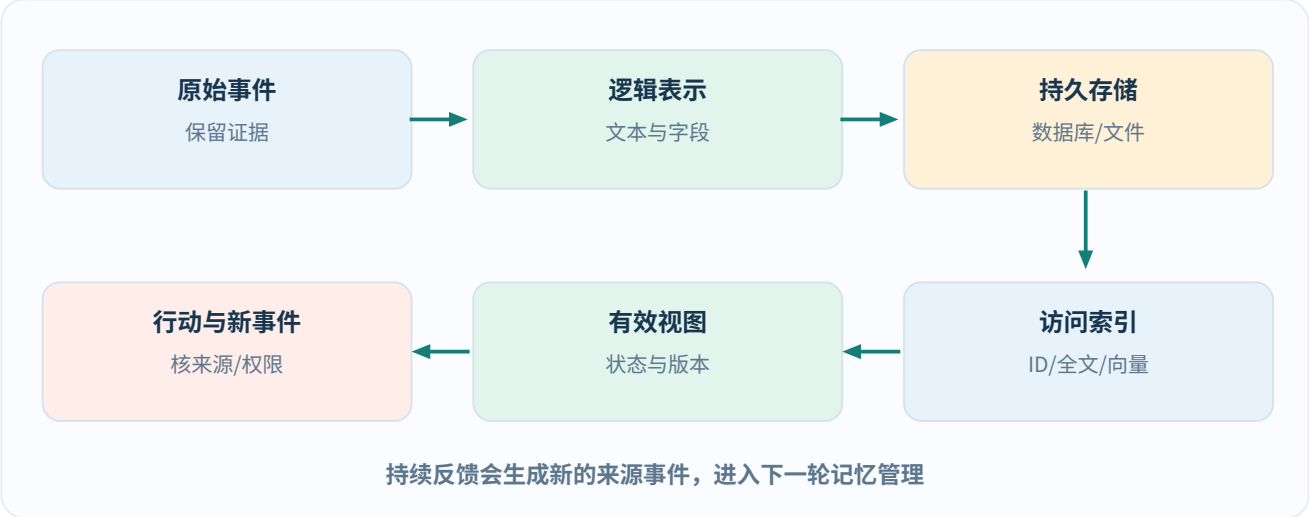


图18　本讲知识地图：表示、存储、检索、版本和决策构成闭环

参考文献与官方资料

[1] Chhikara, P. et al. (2025). Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413. 【原始出处 ↗】

[2] Xu, W. et al. (2025). A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110. 【原始出处 ↗】

[3] PostgreSQL Global Development Group. JSON Types（JSON/JSONB与索引的官方文档）. 【原始出处 ↗】

[4] SQLite Project. SQLite FTS5 Extension（全文索引与虚拟表）. 【原始出处 ↗】

[5] LangChain/LangGraph. Persistence（Checkpointer保存线程状态，Store保存跨线程的应用数据）. 【原始出处 ↗】

[6] W3C. PROV-O: The PROV Ontology（用于数据实体、活动与责任主体的来源关系）. 【原始出处 ↗】

[7] JSON Schema Organization. JSON Schema（结构化数据格式约束背景；JSON Schema相关规范主页）. 【原始出处 ↗】

[8] PostgreSQL Global Development Group. GIN Indexes（广义倒排索引的机制及JSONB/全文索引应用）. 【原始出处 ↗】

参考阅读说明： [1]、[2]属于智能体记忆研究；[3]、[4]、[5]、[8]属于工程系统与官方技术文档；[6]、[7]是数据描述/验证规范的入口。**讲义中的项目事件、表设计和图解是独立教学抽象，不属于某一篇论文的原始实验结论。**